HOW COMPUTERS LEARN

From Perceptron to Agents

70 years of one idea: learn by adjusting weights

Rob Kraft

1943–1958

The idea is born

1943

McCulloch & Pitts

A neuron described as math: inputs, weights and a threshold.

1958

Rosenblatt’s Perceptron

Weights that learn: guess, check, adjust. The rule in today’s demo.

The hype: machines that would soon **see, talk and think**.

1969

The wall: one layer isn’t enough

Minsky & Papert’s *Perceptrons*: a single layer can’t learn XOR.

No single straight line separates the 1s from the 0s.

Hidden layers could fix it, but no one knew how to train them.

XOR: “one or the other, not both”

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

ABOUT 1974–1980

The first AI winter

Promises outran results

Machines that would “think” were years away, not months.

Hardware too weak

Slow computers, tiny memory, almost no digital data.

Funders pulled out

UK’s Lighthill Report (1973) and US DARPA cutbacks.

It froze funding for **all** of AI, not just neural networks.

THE 1980S

Two comebacks

Rules written by people

Expert systems

Experts’ knowledge typed in as thousands of if-then rules. A big commercial boom.

Weights learned from data

Backpropagation

Werbos (1974); Rumelhart, Hinton & Williams (1986). Hidden layers can finally be trained.

ABOUT 1987–1993

The second AI winter

Rules didn’t scale

Expert systems were brittle, costly to maintain, and couldn’t learn.

Markets collapsed

AI hardware sales crashed (1987). Japan’s Fifth Generation project fell short.

Neural nets stalled

Deep networks wouldn’t train. Little data, slow hardware.

Same pattern: **hype outran results**, and funding was cut again.

1989–2005

Quiet progress

1989–1998: LeCun’s CNNs read ZIP codes, then bank checks.

1997: LSTM gives networks a memory for sequences.

1998: MNIST becomes the standard handwritten-digit benchmark.

But…

Support vector machines and random forests won most problems.

Real systems were pipelines of specialized parts.

MISSING INGREDIENT #1

GPUs: built for games, repurposed

1999

Graphics

Thousands of small cores doing the same math at once, to draw 3D games.

→

2007

Programmable

NVIDIA's CUDA lets any program use that math, not just graphics.

→

2010s

Repurposed

Bitcoin mining, then neural networks (AlexNet, 2012: two gaming GPUs).

A neural network is mostly **weighted sums**: exactly the math GPUs do in parallel.

MISSING INGREDIENT #2

The internet: humanity's writing, online

Decades of web pages

Articles, forums, documentation, code.

Wikipedia (2001)

Free, edited knowledge in many languages.

ImageNet (2009)

14 million web images, labeled by people.

Common Crawl (2008 on)

Free snapshots of billions of web pages.

GPT-3 (2020) drew **about 60%** of its training text from Common Crawl.

2006–2012

Deep learning returns

2006: Hinton shows deep networks *can* be trained.

2009–2012: GPUs plus big labeled data (ImageNet).

2010–2012: Speech recognition switches to neural nets.

2012 · AlexNet on ImageNet

15%

error, vs. 26% for the next-best entry

INSIDE A CNN · IMAGE RECOGNITION

Layers build richer understanding

Each layer finds patterns in the layer below. Nobody writes the rules: the network learns them.

Early layers

- edges
- lines
- curves

→

Middle layers

- corners
- shapes
- textures

→

Higher layers

- eyes, ears
- faces
- wheels

→

Final layer

Cat 97%

Dog 2%

Rabbit 1%

Still everywhere: face unlock, medical scans, car cameras, factory inspection.

2017 · “ATTENTION IS ALL YOU NEED”

The transformer: attention

Before: RNNs and LSTMs

Read one word at a time, in order. Slow to train, and early words fade.

Transformer: self-attention

Every word looks back at all earlier words at once. Runs in parallel on GPUs, so models can scale.

“The **trophy** didn’t fit in the suitcase because it was too **big**.”

2018–2022

Scale

2018

GPT and BERT: transformers trained on large text collections

2020

GPT-3: 175 billion weights

2020

Scaling laws: more size, data and compute give predictably better results

Nov 2022

ChatGPT: an LLM tuned into an assistant with human feedback

The demo has **70** weights. GPT-3 has **175,000,000,000**.

THE BIG SHIFT

Attention made scale possible. Scale made in-context learning appear.

Old way

New task → new training → new weights

New way

Frozen weights; describe the task in the prompt

2020–2024 · AFTER LLMS

Giving a frozen model facts and hands

Once trained, the weights don’t change. So we change **what goes into the prompt**.

RAG (2020) · look things up

Facts

Search your documents and paste the relevant parts into the prompt.

Covers the knowledge cutoff and your private data.

Tools (2023) · MCP (2024)

Hands

The model asks for a call, get_weather(“Kansas City”); ordinary code runs it.

MCP: one standard plug for any tool or data source.

2022–2025

Models that think first

2022 · a prompting trick

Chain-of-thought

Ask for step-by-step work and more answers come out right.

→

2024 on · trained in

Reasoning models

Rewarded for checkable right answers, they learn to try, check and back up.

“A bat and a ball cost $1.10. The bat costs $1 more than the ball. What does the ball cost?”

Quick answer: 10¢ (wrong). Step by step: **5¢**.

2025

Agents and context engineering

The agent loop

1. **Think:** the model picks an action
2. **Act:** the harness runs the tool
3. **Observe:** the result joins the context
4. **Repeat** until the goal is met

Context engineering

What the model sees

The context window is its only working memory. Each turn, choose the instructions, tools, documents and memory that go in.

Too little: missing facts. Too much: cost and distraction.

Claude Code and Codex work this way: read files, edit, run tests, read failures, fix, repeat.

THE THROUGH-LINE

Same idea, new ingredients

Perceptron → Deep networks → Transformer → LLM → Agent

Why the winters happened: hype outran results, and a key ingredient was missing: data, compute, or a way to train.

What’s missing today?

TRY IT YOURSELF

Train your own neural network

Scan to open the Perceptron Playground in your browser. Nothing to install.

robkraft.github.io/neural-networks-talk

Source code: github.com/RobKraft/neural-networks-talk

